



Vulnerability Assessment & Penetration Testing Report

Target:

<https://mstu.offenso.local/>

Prepared For:

Hackware Security Solutions LLP

Prepared By:

Basith Mohammed Ali

Date:

9 July 2026

Version:

1.0

Table of Contents

1. Executive Summary	3
2. Scope	3
3. Methodology	3
4. Findings Summary	4
F-01 – Network Enumeration	5
F-02 – Exposure of Correct Exam Answers via Client-Side API Response	6
F-03 – Business Logic Flaw in Attendance Management	7
F-04 – Stored Cross-Site Scripting (XSS) in Support Ticket	9
F-05 – Information Disclosure via Exposed Internal User Enumeration	10
F-06 – Negative Payment Amount Accepted (Business Logic Flaw)	12
F-07 – Clickjacking via Missing X-Frame-Options Header	14
F-08 – Weak Authentication Rate Limiting	16
Conclusion	17

1. Executive Summary

This Vulnerability Assessment and Penetration Test (VAPT) was conducted on the MS e-Gov Portal to identify security weaknesses that could affect the confidentiality, integrity, and availability of the application. The assessment identified several security issues, including business logic flaws, stored cross-site scripting (XSS), information disclosure, clickjacking, and weak authentication controls. Appropriate remediation is recommended to reduce the overall security risk.

2. Scope

The following target was included within the assessment scope:

Target	Type
https://mstu.offenso.local	Web Application

The assessment focused on authenticated and unauthenticated functionality of the web application.

3. Methodology

Phase	Description
Information Gathering	Collected target information.
Network Enumeration	Identified ports and services.
API Testing	Tested application endpoints.
Authentication Testing	Tested login and sessions.
Business Logic Testing	Tested application workflows.
Validation & Reporting	Verified findings and documented results.

4. Findings Summary

ID	Finding	Severity
F-01	Network Enumeration	Informational
F-02	Exposure of Correct Exam Answers via Client-Side API Response	High
F-03	Business Logic Flaw in Attendance Management	High
F-04	Stored Cross-Site Scripting (XSS) in Support Ticket	Medium
F-05	Information Disclosure via Exposed Internal User Enumeration	Medium
F-06	Negative Payment Amount Accepted (Business Logic Flaw)	Medium
F-07	Clickjacking via Missing X-Frame-Options Header	High
F-08	Insufficient Brute-Force Protection on Authentication Endpoint	Medium

The assessment identified 8 security findings, comprising 2 High, 4 Medium, and 1 Informational finding. Timely remediation is recommended to reduce the overall security risk.

Severity	Count
Critical	0
High	3
Medium	4
Low	0
Informational	1

F-01 – Network Enumeration

Severity	Informational
CVSS v3.1	N/A (Informational)

Description

Network enumeration was performed to identify exposed services, running applications, and potential attack surfaces on the target host.

Command Used

```
nmap -sC -sV -Pn -T4 192.168.56.111
```

Proof of Concept

Perform a service and version scan against the target host using Nmap.

Results

Port	Service	Version
22/tcp	SSH	OpenSSH 9.9p1 (Ubuntu)
80/tcp	HTTP	nginx 1.26.3
443/tcp	HTTPS	nginx 1.26.3
5000/tcp	HTTP	Node.js Express Framework

Observation

The scan identified four accessible services, including SSH, HTTP, HTTPS, and a Node.js Express application running on port **5000**, which became the primary target for subsequent security testing.

F-02 – Exposure of Correct Exam Answers via Client-Side API Response

CVSS v3.1 Score	Severity
8.1	High
Affected Endpoint	https://mstu.offenso.local/api/exams/student/writable

Description

The examination API returns the complete question paper along with the **correctAnswer** value for every question before the examination is completed. Because this information is included in the server response, any authenticated student can intercept the request using browser developer tools or an intercepting proxy and obtain the correct answers without answering the questions legitimately.

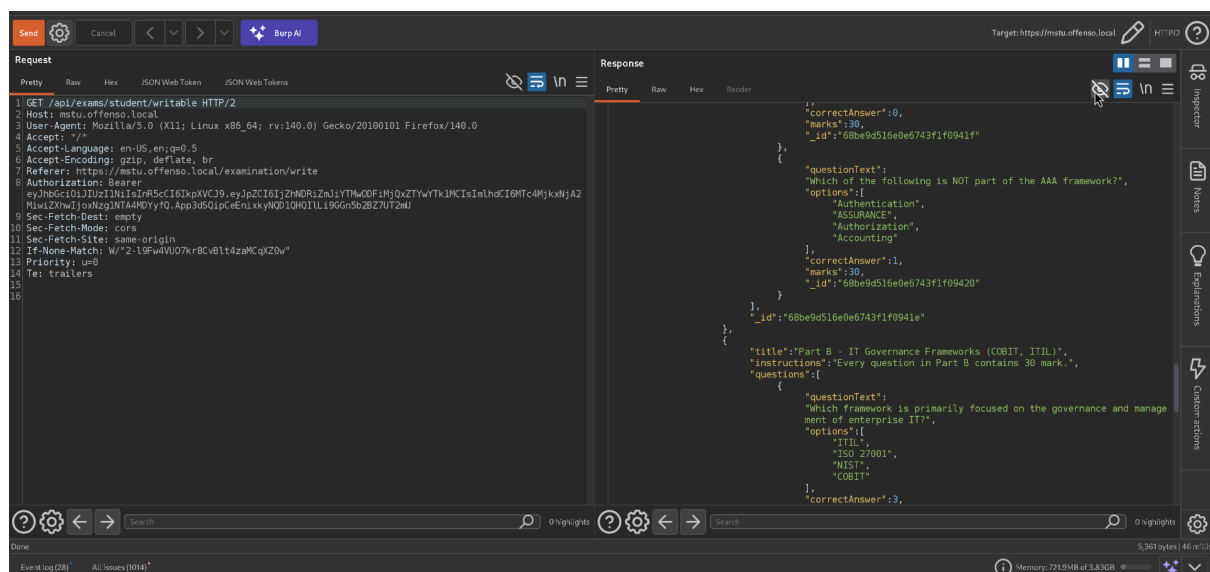
Proof of Concept

1. Log in as a student.
2. Start an examination.
3. Intercept the request:

GET /api/exams/student/writable

Observe the JSON response.

The response contains the answer key within the **correctAnswer** field.



The value `correctAnswer` : reveals the correct option before the student submits the examination.

Impact

An authenticated student can obtain the correct answers before completing the examination, allowing them to achieve artificially high scores. This compromises the integrity and fairness of the examination process and may affect the reliability of published results.

Recommendation

- Do not include the `correctAnswer` field in API responses sent to students.
- Store answer keys only on the server.
- Validate submitted answers server-side after the examination is completed.
- Ensure only administrators or authorized exam evaluators can access answer keys.

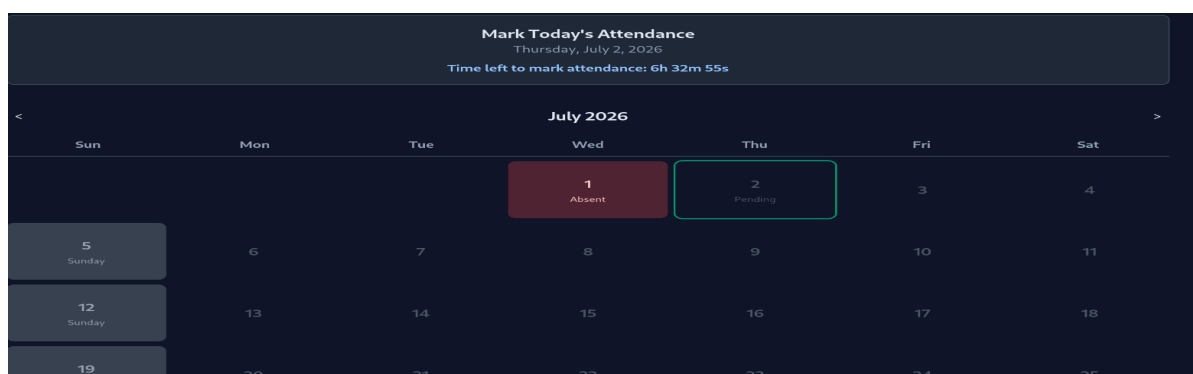
F-03 – Business Logic Flaw in Attendance Management

Severity	High
CVSS v3.1	8.1
Affected Endpoint	https://mstu.offenso.local/api/attendance/mark

Description

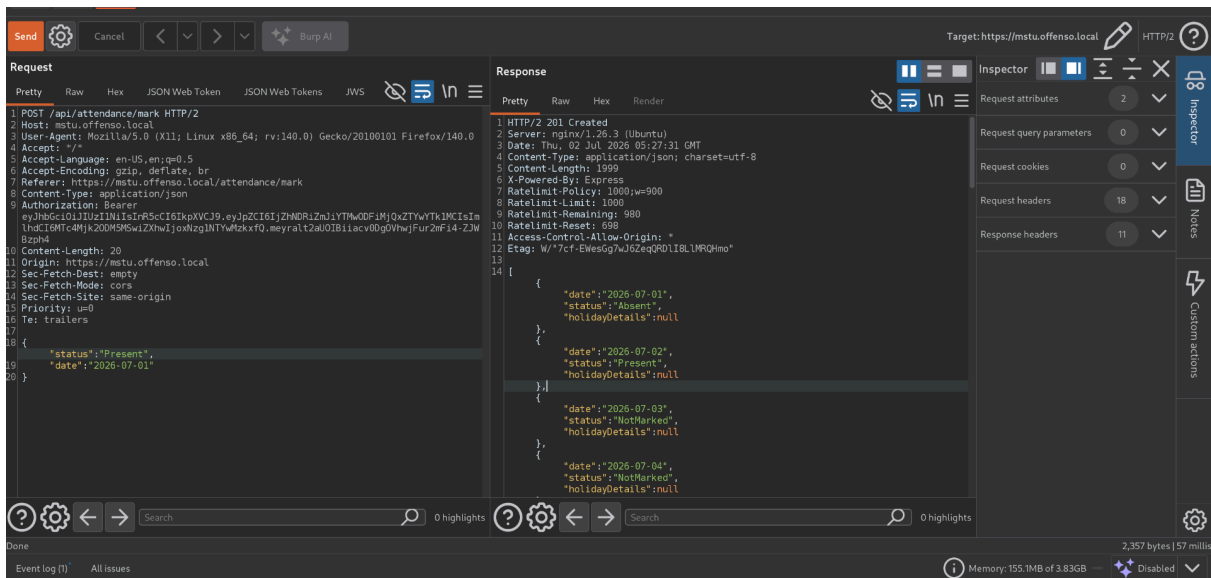
The attendance marking functionality fails to enforce server-side validation of attendance dates. An authenticated student can intercept the attendance request and modify the `date` parameter to mark attendance for a previous day.

This allows unauthorized modification of historical attendance records, compromising the integrity of attendance data.

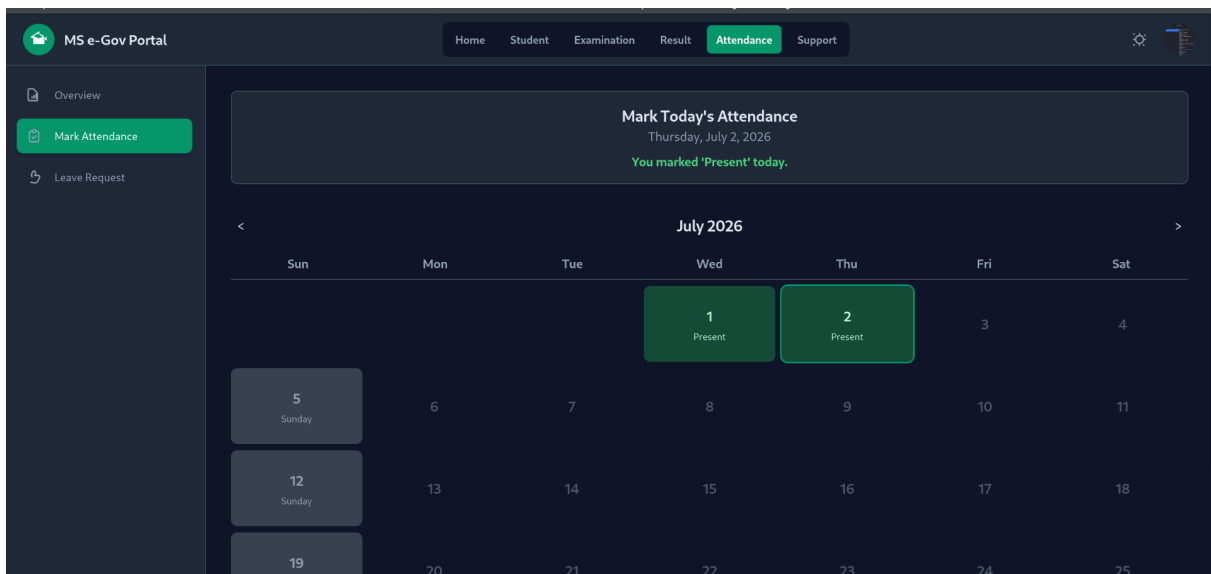


Proof of Concept

1. Authenticate to the application using a valid student account.
2. Navigate to **Attendance** → **Mark Attendance**.
3. Intercept the **POST /api/attendance/mark** request using Burp Suite.
4. Modify the JSON request body, setting the **date** parameter to a prior date (e.g., **2026-07-01**) instead of the current date, with **status** set to **"Present"**:



5. Forward the modified request to the server.
6. Navigate back to the Attendance calendar view in the browser and refresh the page.
7. The attendance status for **01 July 2026** changes from **Absent** to **Present**.



Impact

An authenticated student can alter historical attendance records by modifying the attendance date within the request. This compromises the integrity of attendance data and enables unauthorized manipulation of academic records, potentially affecting attendance tracking and institutional reporting.

Recommendation

Validate attendance dates on the server side and permit attendance marking only for the current date within the authorized attendance window. Reject any requests containing modified or historical dates and implement audit logging to record attendance modifications.

F-04 – Stored Cross-Site Scripting (XSS) in Support Ticket

Severity	Medium
CVSS v3.1	5.4
Affected Endpoint	PUT /api/support/{ticket_id}/reply

Description

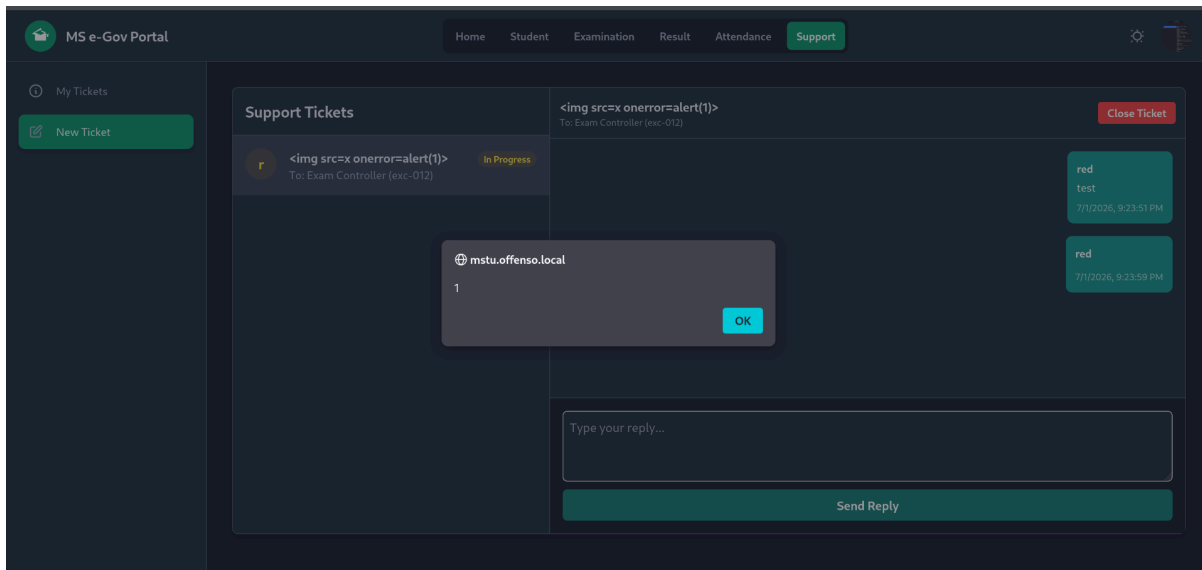
The support ticket functionality does not properly sanitize user-supplied input. An authenticated user can inject malicious JavaScript into the ticket subject or message, which is stored by the application and executed when the ticket is viewed.

Proof of Concept

1. Log in as a student.
2. Navigate to **Support** → **New Ticket** and create a support ticket.
3. Open the created ticket.
4. Enter the following payload into the **Reply** field:

`<img src=x onerror=alert(1)>`

5. Click **Send Reply**.
6. Open or refresh the ticket conversation.
7. The payload is rendered and JavaScript executes, resulting in a browser alert.



Impact

An attacker can execute arbitrary JavaScript in the context of the application. Successful exploitation may lead to session hijacking, credential theft, unauthorized actions on behalf of users, phishing attacks, or defacement of application content.

Recommendation

Validate and sanitize all user-supplied input before storing it and encode output before rendering it in the browser. Implement a strong Content Security Policy (CSP) to reduce the impact of XSS attacks.

F-05 – Information Disclosure via Exposed Internal User Enumeration

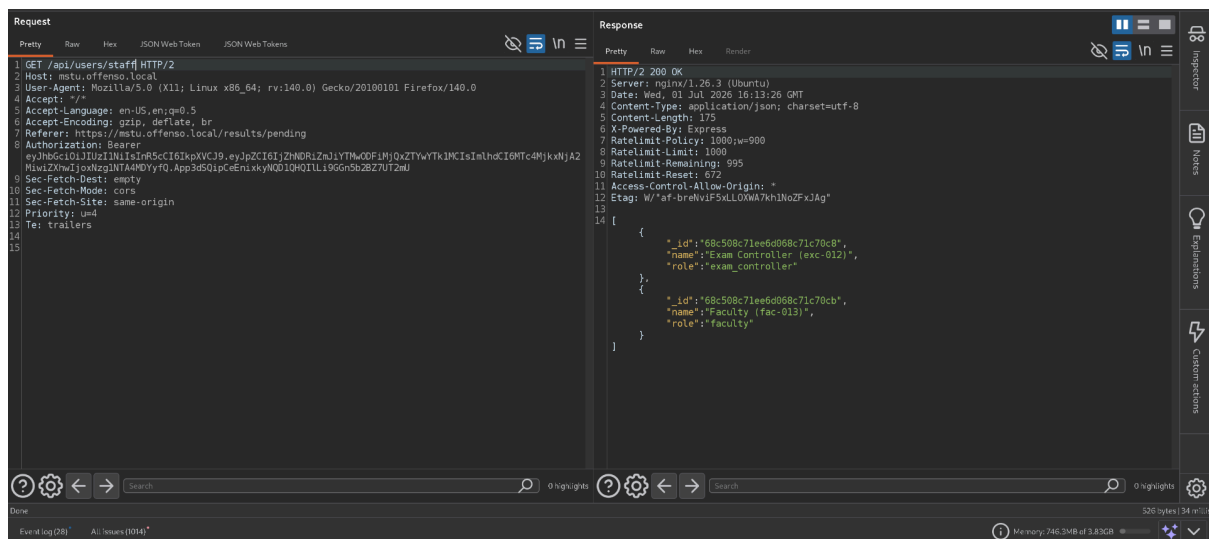
Severity:	Medium
CVSS v3.1	5.3
Affected Endpoint	GET /api/users/staff

Description

The application exposes information about internal users through an API endpoint. An authenticated student can retrieve internal staff names, roles, and unique identifiers without authorization.

Proof of Concept

1. Log in as a student.
2. Intercept or browse to the exposed API endpoint.
3. Observe that the server returns internal user information, including:
 - Staff names
 - Roles
 - Internal user IDs
4. No additional authorization checks are performed before returning the data.



Impact

An attacker can enumerate internal users and their roles within the application. This information may assist in targeted phishing, social engineering, user enumeration, or identifying privileged accounts for further attacks.

Recommendation

Restrict access to internal user information using server-side authorization. Return only the minimum information required for the application's functionality and avoid exposing internal identifiers to unauthorized users.

F-06 – Negative Payment Amount Accepted (Business Logic Flaw)

Severity	Medium
CVSS v3.1	6.5
Affected Endpoint	POST /api/payments/initiate

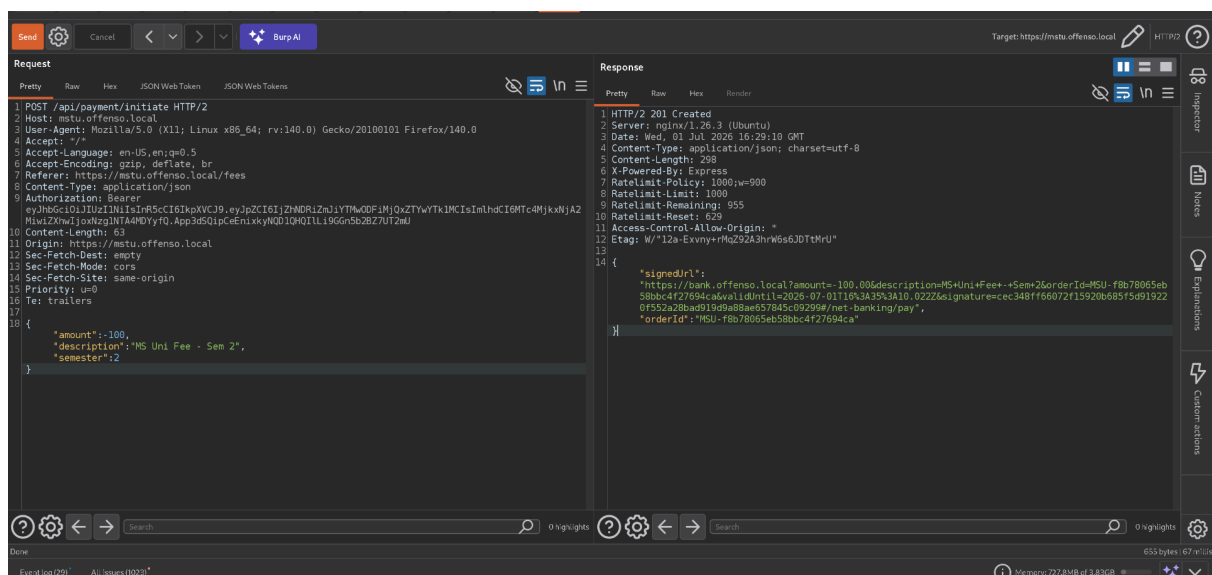
Description

The application fails to validate the payment amount on the server side. By intercepting the payment initiation request and modifying the `amount` parameter to a negative value, the application generated a valid payment session and processed the transaction successfully.

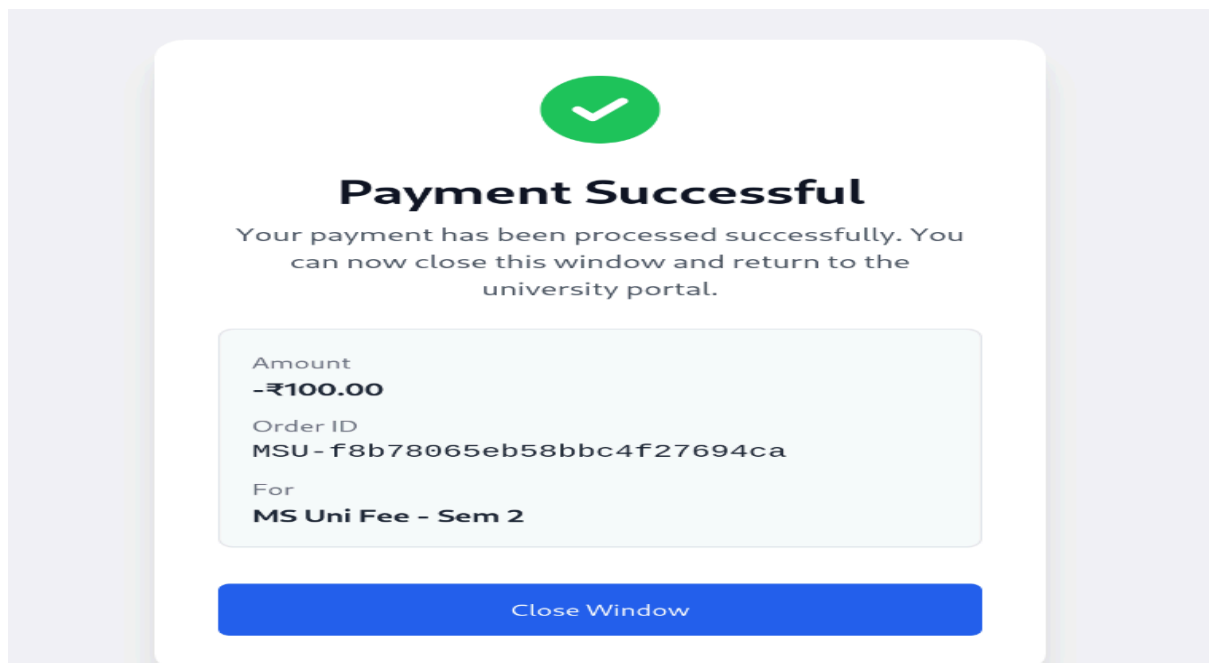
As a result, the student's payment record was updated with a negative payment amount, resulting in incorrect fee calculations.

Proof of Concept

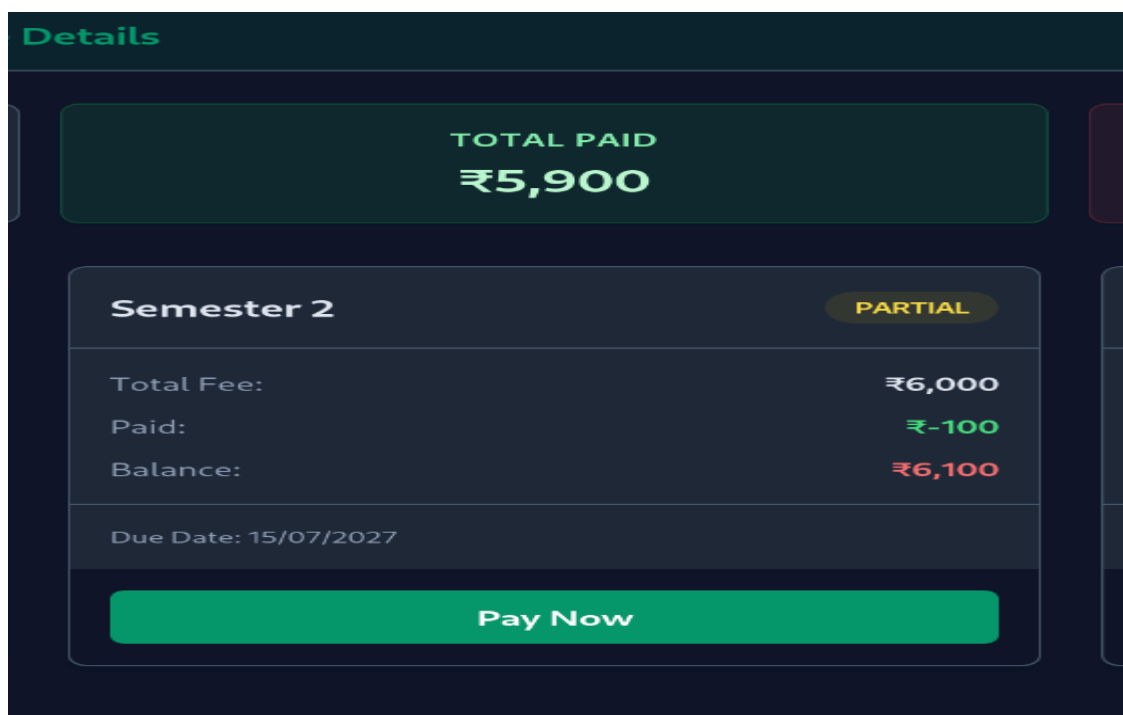
Step 1: Intercept the payment request and modify the **amount** parameter from its original value to **-100**. Forward the modified request.



Step 2: Open the returned `signedUrl` in the browser. The payment page displays a negative payment amount.



Step 3: Complete the payment and verify the student's fee details. The application records the payment as **-₹100**, resulting in an incorrect balance.



Impact

The application accepts manipulated payment amounts, allowing users to submit negative values. This can result in incorrect fee records, inaccurate financial calculations, and may be abused to manipulate payment transactions.

Recommendation

Implement strict server-side validation to ensure that payment amounts are positive and match the actual fee amount. The server should reject negative or modified values and calculate payment amounts internally rather than trusting client-supplied input.

F-07 – Clickjacking via Missing X-Frame-Options Header

Severity	High
CVSS v3.1	7.4
Affected Endpoint	https://mstu.offenso.local/login

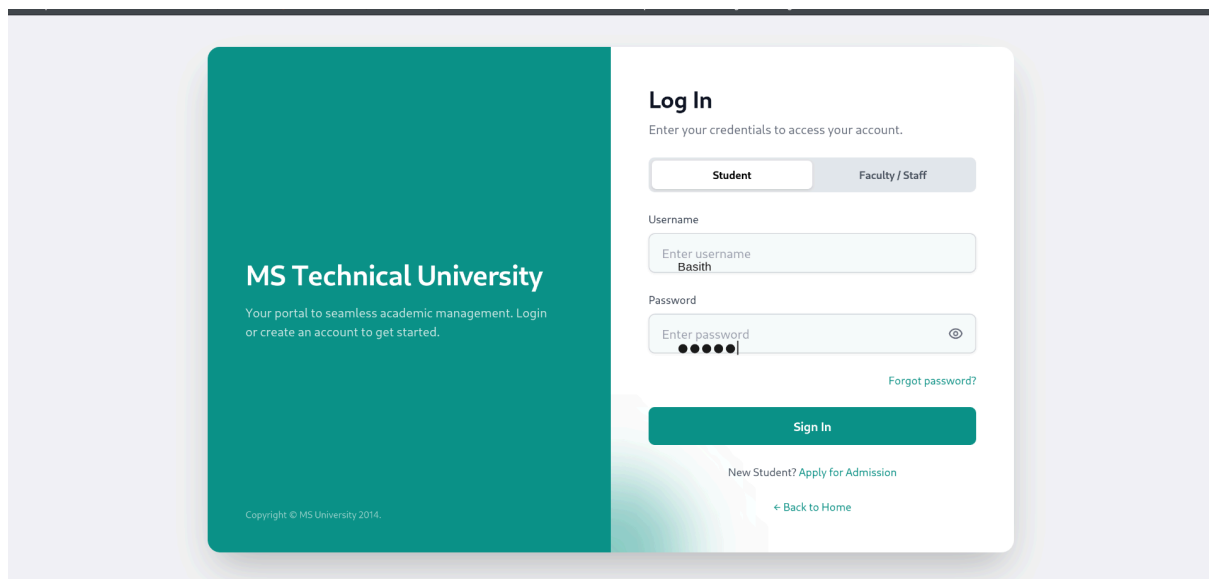
Description

The application does not implement the `X-Frame-Options` or `Content-Security-Policy: frame-ancestors` HTTP response header on the login page. This allows an attacker to embed the target application inside an iframe on a malicious external page. By overlaying invisible input fields precisely over the legitimate login form, the attacker can silently capture victim credentials without any visible indication of compromise.

Proof of Concept

Step 1: Create a malicious HTML page that embeds <https://mstu.offenso.local/login> inside a fullscreen iframe. The target page loads successfully, confirming the absence of framing protections.

Step 2: Overlay transparent input fields precisely over the real username and password fields. The victim sees and interacts with what appears to be the legitimate login page.



Step 3: The victim enters their credentials into the invisible overlay fields. All keystrokes are silently captured and exfiltrated to the attacker-controlled server in real time.

Step 4: The attacker's server receives the captured credentials as shown in the proof of concept screenshots.

```
[+] CAPTURED:
    event : username
    value : Basith

[+] CAPTURED:
    event : password
    value : A

[+] CAPTURED:
    event : password
    value : Ad

[+] CAPTURED:
    event : password
    value : Adm

[+] CAPTURED:
    event : password
    value : Admi

[+] CAPTURED:
    event : password
    value : Admin
```

Impact

An attacker can host a convincing replica of the application by embedding the legitimate site inside an iframe. Victims interacting with the page unknowingly submit their credentials directly to the attacker. This can lead to full account takeover, unauthorized access to student records, financial data, and sensitive academic information.

Recommendation

Add the following HTTP response headers to all application pages, particularly the login page:

X-Frame-Options: DENY
Content-Security-Policy: frame-ancestors 'none'

This prevents the application from being embedded inside iframes on external origins, fully mitigating the clickjacking risk.

F-08 – Weak Authentication Rate Limiting

Severity	Medium
CVSS v3.1	5.3
Affected Endpoint	POST /api/users/login

Description

The login endpoint implements a rate limit of **1000 authentication requests per 900 seconds (15 minutes)**, which is excessively permissive for an authentication mechanism. During testing, no account lockout or additional authentication controls were observed after multiple failed login attempts, increasing the risk of brute-force attacks.

Proof of Concept

Step 1: Capture the **POST /api/users/login** request and send it to Burp Suite Intruder.

Step 2: Launch a password attack against the same user account using a password wordlist.

Step 3: After multiple failed attempts, request **#99** using the password **Password1** returned **HTTP/2 200 OK** with a valid JWT token, confirming successful authentication without account lockout.

Step 4: The response headers indicate a limit of **1000 requests per 900 seconds**, allowing a high number of authentication attempts.

The screenshot displays the Burp Suite Intruder interface. The top section shows a list of requests, with request #99 highlighted. The bottom section provides a detailed view of request #99, showing the raw HTTP request and response.

Request #99 Details:

- Method: POST
- URL: /api/users/login
- Status: 200
- Response received: 5006
- Length: 780
- Comment: Contains a JWT

Request Headers:

```
Content-Length: 69
Origin: https://msfu-offenso.local
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin
Priority: uo
Te: trailers
Connection: keep-alive
```

Response Headers:

```
Content-Length: 1808
X-Powered-By: Express
RateLimit-Policy: 1000/w=900
RateLimit-Limit: 1000
RateLimit-Remaining: 863
RateLimit-Reset: 362
Access-Control-Allow-Origin: *
Etag: W/"710-QKq102JAaGrAJb1d1Np8019tq2s"
```

Response Body (JSON):

```
{
  "user": {
    "id": "6a41f783a3081b241e5e9d9f",
    "username": "0HA25J299",
    "password": "Password1",
    "loginRole": "student"
  }
}
```


Impact

An attacker can perform automated brute-force or password-spraying attacks, increasing the likelihood of unauthorized account access through weak or reused passwords.

Recommendation

Reduce the authentication rate limit, implement temporary account lockout or progressive delays after repeated failed login attempts, and consider CAPTCHA or multi-factor authentication (MFA) for additional protection.

Conclusion

The VAPT assessment identified multiple security vulnerabilities affecting the MS e-Gov Portal, including business logic flaws, information disclosure, cross-site scripting (XSS), clickjacking, and weak authentication controls. Addressing the recommended remediation measures will significantly improve the application's security posture and reduce the risk of unauthorized access, data exposure, and business impact.

End of Report